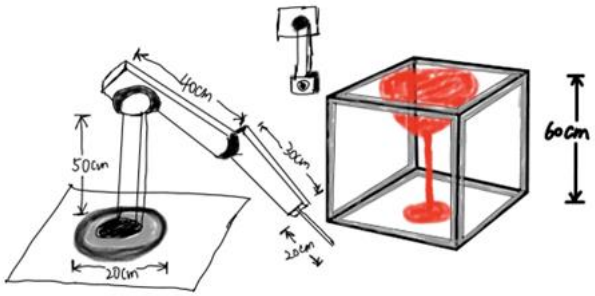
Dynamic Equations of Motion - calculations:

In this project, the model I applied is consistent with previous steps, which is shown in the figure.



To calculate its Euler-Lagrange equations, based on the knowledge in class, the torque of end effector is gained by:

$$D(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau,$$

where $q = [q_1 \dots q_n]^T$, $\dot{q} = [\dot{q}_1 \dots \dot{q}_n]^T$, $\ddot{q} = [\ddot{q}_1 \dots \ddot{q}_n]^T$, $\tau = [\tau_1 \dots \tau_n]^T$ are the vectors of generalized positions, velocities, accelerations, and forces (torques), respectively, $D(q) \in R^{n*n}$ is the matrix of inertia, $C(q,\dot{q}) := \sum_{j=1}^{n} cijk(q)\,\dot{q}j$ is the matrix of the centrifugal and Coriolis forces, and

$$G(q) = \left[\frac{\partial P(q)}{\partial q_1} \quad \dots \quad \frac{\partial P(q)}{\partial q_n}\right]^T$$

is the vector of potential (gravity) forces.

In this model, we assume that the mass of first joint m_1 is 2kg, with a length of 0.4m. The mass of second joint m_2 is 1kg, with a length of 0.3m. And the first joint has an initial joint angle of pi/4, and the second joint has an initial joint angle of pi/6, while the initial joint angular velocities are 0.2rad/s and 0.1 rad/s respectively.

These variables are changeable in different situation. After calculation, the final torque is

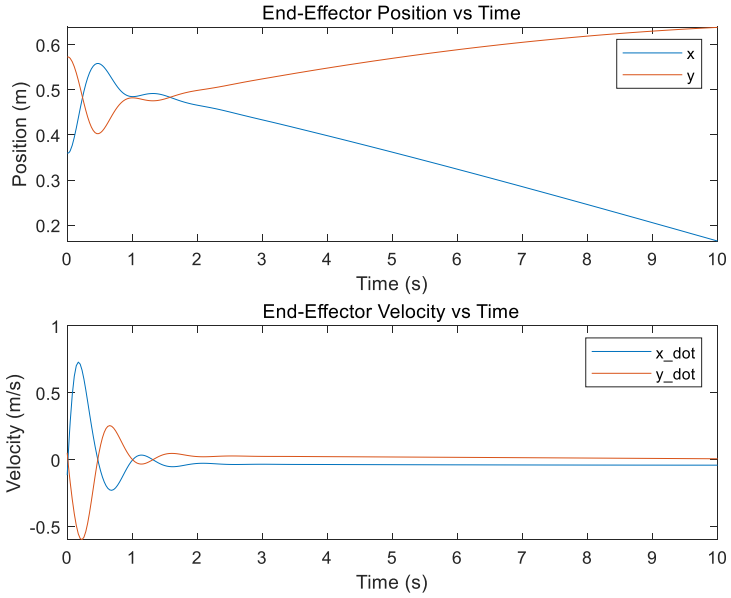
```
>> yzj1
joint torque:
          (2943*sin(q1 + q2))/1000 + (8829*sin(q1))/1000
(2943*sin(q1 + q2))/1000 - (9*q1_dot*q2_dot*sin(q2))/100
```

,from the simulated formulation in Matlab. Specifically, in this condition, the torque will be:

```
numerical torque:
       (20601*2^(1/2))/4000 + (2943*6^(1/2))/4000
(2943*2^(1/2))/4000 + (2943*6^(1/2))/4000 - 9/400
```

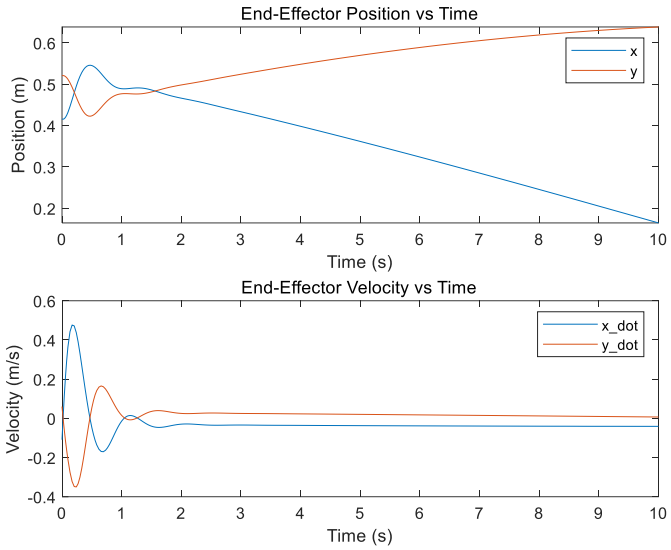
Dynamic Equations of Motion - implementation:

From the outcome in last part, we can define a function that includes all the inputs we need in the initial condition, and the output demonstrate the robot motion (positions and velocities). For example, keeping the same condition as before, the output is shown in the picture:



(a)Condition1: joint angle 1 =pi/4,joint angle2= pi/6, joint angular velocity1 = 0.2rad/s , joint angular velocity2=0.1 rad/s

Choosing another condition, i.e., the first joint has an initial joint angle of pi/5, and the second joint has an initial joint angle of pi/5, while the initial joint angular velocities are 0.1rad/s and 0.2 rad/s respectively. This time the result was like this:



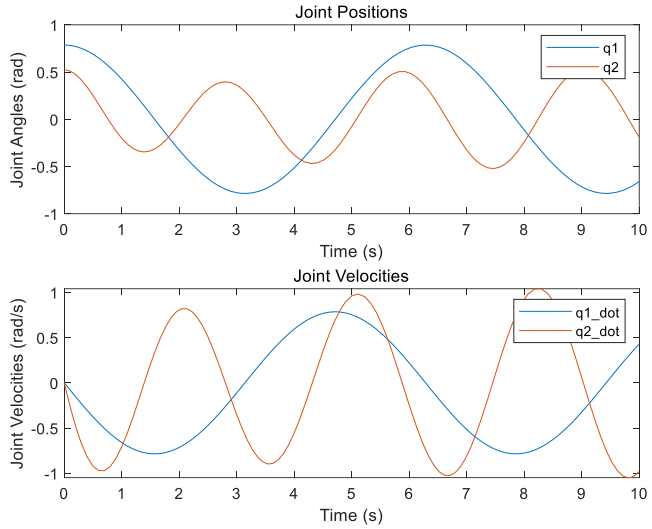
(b)Condition2: joint angle 1 =pi/5,joint angle2= pi/5, joint angular velocity1 = 0.1rad/s , joint angular velocity2=0.2 rad/s

The two results are similar but exists slightly differences. The plots show that the robotic system exhibits initial oscillations in both position and velocity due to transient dynamics. Over time, xx decreases and yy increases steadily, while the velocities stabilize around zero. This behavior aligns with expected dynamics under applied torques, demonstrating a realistic system response.

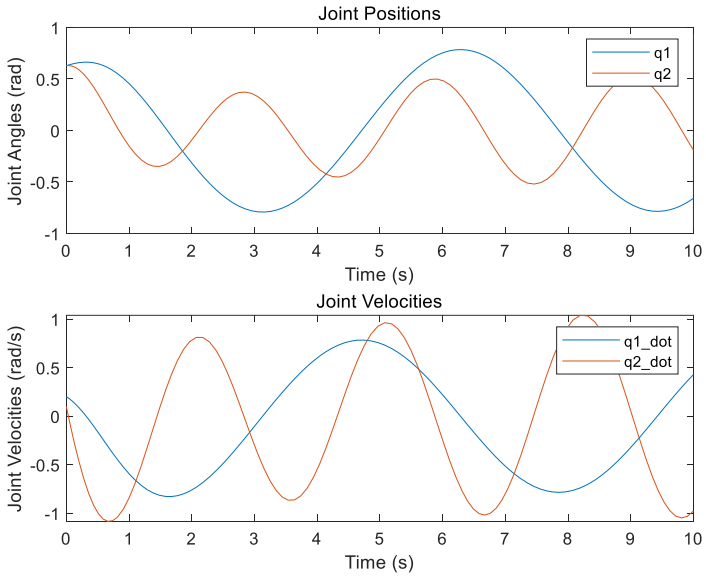
Control System Design:

In the previous part, the derived equations were implemented in MATLAB using symbolic computation for accuracy. An inverse dynamics controller was implemented to achieve trajectory tracking. The control strategy employs a Proportional-Derivative (PD) controller combined with the inverse dynamics model. Desired joint trajectories are time-varying sinusoidal functions with their derivatives used for velocity and acceleration tracking. Numerical simulation is performed using MATLAB's ODE45 solver, integrating the system's dynamics over time. Initial conditions include joint angles, velocities, and predefined trajectory functions. The control gains (K_p and K_d) regulate position and velocity errors. The outputs are joint positions and velocities over time, illustrating the system's capability to track the desired trajectories accurately while compensating for dynamic effects such as inertia, Coriolis forces, and gravity. This approach demonstrates the effectiveness of combining inverse dynamics with feedback control for nonlinear robotic systems.

Here are the results for the previous two conditions, respectively.



(a)Condition1: joint angle 1 =pi/4,joint angle2= pi/6, joint angular velocity1 = 0.2rad/s , joint angular velocity2=0.1 rad/s

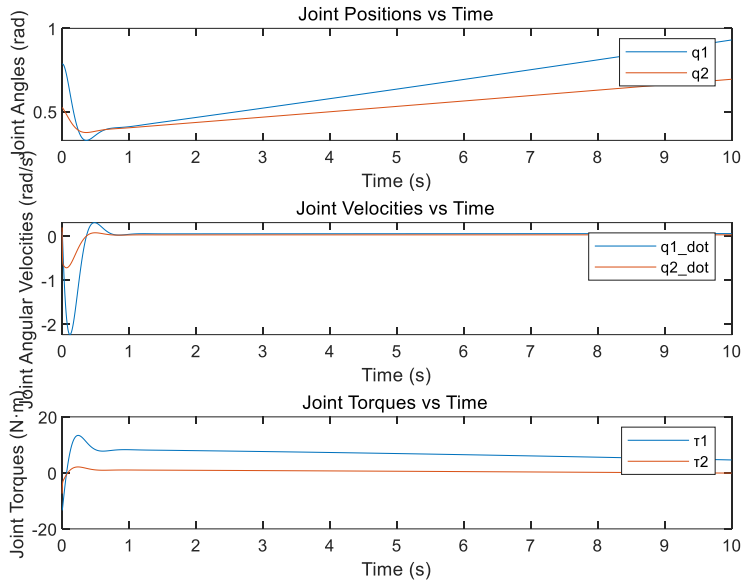


(b)Condition2: joint angle 1 =pi/5,joint angle2= pi/5, joint angular velocity1 = 0.1rad/s , joint angular velocity2=0.2 rad/s

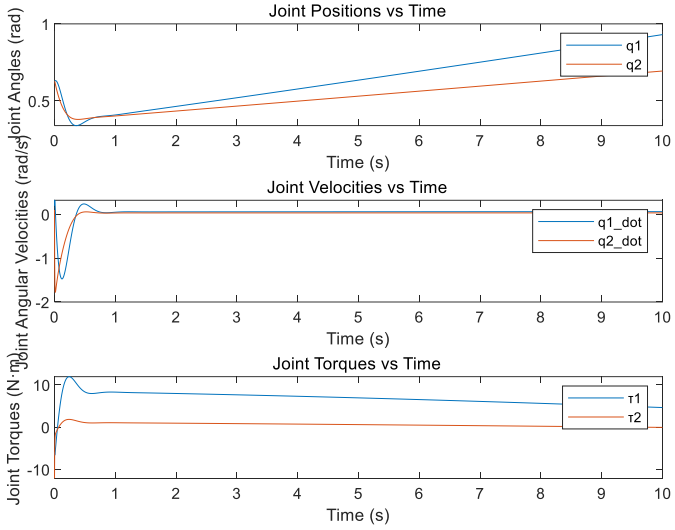
Control System Simulation:

The trajectory defines the desired joint positions, velocities, and accelerations over time. Using inverse dynamics, the required joint torques are calculated to compensate for inertia, Coriolis, and gravitational forces. A time scale is incorporated to determine motion velocity, and MATLAB's numerical solver (ODE45) integrates the equations of motion. The results include plots for joint positions, velocities, and torques, providing

insights into the system's behavior. The analysis evaluates whether the controller ensures smooth tracking with minimal oscillations and validates torque efficiency under dynamic conditions. And the outcomes are as follows:



(a)Condition1: joint angle 1 =pi/4,joint angle2= pi/6, joint angular velocity1 = 0.2rad/s , joint angular velocity2=0.1 rad/s



(b)Condition2: joint angle 1 =pi/5,joint angle2= pi/5, joint angular velocity1 = 0.1rad/s , joint angular velocity2=0.2 rad/s